ECE3300L Sine FFN Project

Pretrained int8 Feed-Forward Network Inference on the Nexys A7

Project direction

Implement inference only for the pretrained TensorFlow Lite Micro Hello World sine model. A computer will quantize an input value, send it to the Nexys A7 over UART, and dequantize the returned prediction. The FPGA must evaluate the model using synthesizable RTL, fixed integer parameters, and a reusable multiply-accumulate datapath. Training or changing the model is not part of this project.

What you will build

1. A model-preparation flow that obtains the official pretrained `hello_world_int8.tflite` file and records its exact tensor contract.
2. An export utility that converts weights, biases, scales, zero-points, and multiplier/shift constants into RTL-readable initialization files.
3. A signed dense-layer datapath containing an int8 multiplier, an int32 accumulator, requantization, saturation, and optional ReLU.
4. An FFN controller that executes the fixed 1 -> 16 -> 16 -> 1 topology by reusing one physical MAC unit.
5. A UART bridge that accepts a quantized input byte and returns a quantized output byte using fixed request and response frames.
6. A host reference application that quantizes x, communicates with the board, dequantizes y, and compares the result with the TensorFlow Lite model.

Fixed project configuration

Item	Required configuration
Target board	Digilent Nexys A7; 100 MHz board clock
Model	TensorFlow Lite Micro Hello World pretrained int8 model
Topology	Scalar input -> Dense(16, ReLU) -> Dense(16, ReLU) -> Dense(1, linear)
Function	Approximate y = sin(x) over the model's trained input interval, nominally 0 to 2pi
Arithmetic	int8 weights and activations; int32 biases and accumulators
Datapath	One serial MAC reused by all neurons
Host link	UART, 115200 baud, 8 data bits, no parity, 1 stop bit

Lab objectives

- Translate a trained neural-network graph into an explicit RTL datapath and controller.
- Preserve a pretrained model's tensor ordering and quantization contract.
- Apply signed arithmetic, accumulator width growth, ROM initialization, and cycle-based control.
- Separate model inference latency from UART communication time.

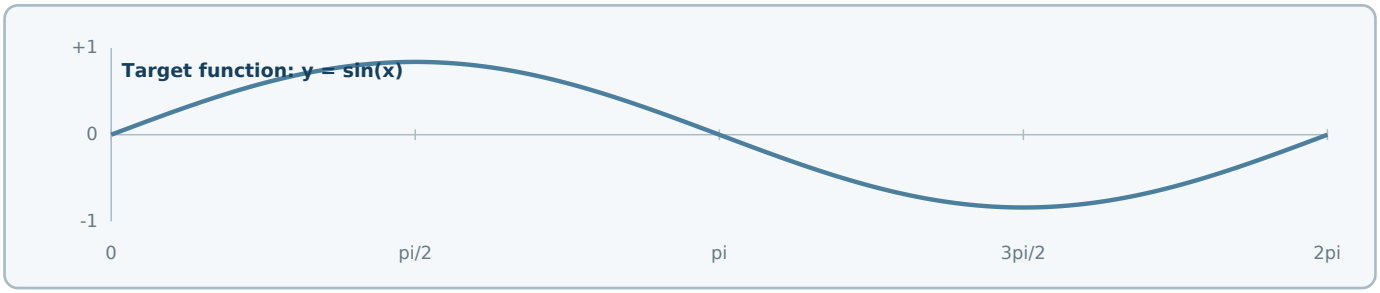
The model you are implementing

A feed-forward network is a sequence of dense layers. Each neuron forms a weighted sum, adds a trained bias, and optionally applies an activation. For this assignment the structure and parameters are fixed; the FPGA only performs the forward pass.

The FPGA evaluates the same fixed topology for every UART request.

```
h1 = ReLU(W1 x + b1)
h2 = ReLU(W2 h1 + b2)
y = W3 h2 + b3
```

Layer	Input -> output	Weights	Biases	Activation
Dense 1	1 -> 16	16	16	ReLU
Dense 2	16 -> 16	256	16	ReLU
Dense 3	16 -> 1	16	1	Linear
Total		288	33	321 trained parameters



What the FPGA is - and is not - doing

The network does not contain an equation for sine. Its 321 trained parameters shape a piecewise approximation. Your RTL must preserve those parameters and the model's integer arithmetic. It does not learn, update weights, or calculate a trigonometric series.

Software model and hardware architecture

Model concept	RTL realization
Dense layer	Controller repeatedly addresses parameter ROM and feeds a shared MAC
Neuron	Bias initialization followed by a sequence of multiply-accumulate cycles
ReLU	Clamp the quantized output below the real-zero code
Layer tensor	A 16-entry signed activation register bank
Forward pass	A deterministic FSM sequence ending with a one-cycle done pulse

Model setup: obtain and identify the model

Use the existing int8 model distributed with TensorFlow Lite Micro. Do not run the training script. Record the repository commit so the exact model file can be reproduced later.

1. Create an isolated Python environment

```
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install tensorflow numpy
# On Windows, use .venv\Scripts\python in place of .venv/bin/python.
```

2. Obtain the official model

```
git clone https://github.com/tensorflow/tflite-micro.git
git -C tflite-micro rev-parse HEAD
MODEL=tflite-micro/tensorflow/lite/micro/examples/hello_world/models/hello_world_int8.tflite
```

3. Inspect the graph before extracting tensors

- Open the `.tflite` file in Netron and identify the three fully connected operators, tensor shapes, and fused activations.
- Load the model with `tf.lite.Interpreter`, call `allocate_tensors()`, then collect `get_input_details()`, `get_output_details()`, and `get_tensor_details()`.
- Use each operator's input tensor indices to associate a weight matrix and bias vector with the correct layer. Do not rely only on tensor-name substrings.
- Confirm the topology against the model itself. If a newer repository revision differs from 1 -> 16 -> 16 -> 1, use the fixed model revision specified by the instructor rather than silently changing the RTL.

Required model contract record

Create a machine-readable manifest and a short human-readable table containing all of the following:

For each tensor or operator	Record
Model input and output	Tensor index, name, shape, dtype, scale, and zero-point
Weight matrix	Layer association, shape, storage order, int8 values, scale array, zero-point array, and quantized dimension
Bias vector	Shape, int32 values, scale array, and zero-point
Intermediate activation	Shape, dtype, output scale, output zero-point, and fused activation
Model identity	Source URL, repository commit, file size, and SHA-256 hash

Model lock

After the manifest and hash are recorded, treat the `.tflite` file as read-only. The exported RTL constants, software reference, and final bitstream must all originate from this one file.

Model setup: reference inference and export

Before writing RTL, establish an integer software reference. This separates model-conversion issues from hardware-control issues and gives the RTL an exact result to reproduce.

4. Establish the software reference

- Sweep representative x values across 0 to 2pi, quantize them with the model input scale and zero-point, invoke the int8 interpreter, and save both the raw int8 outputs and dequantized values.
- Include boundary values, zero crossings, positive and negative peaks, and enough intermediate points to reveal the model's approximation shape.
- Save a compact set of raw input/output byte pairs as golden vectors for later RTL simulation. The raw int8 result is the authoritative comparison value.

5. Export the fixed model constants

Required file	Contents
layer1_weights.mem	16 int8 values in the exact order consumed by Dense 1
layer1_bias.mem	16 int32 bias values
layer2_weights.mem	256 int8 values with a documented neuron-major address mapping
layer2_bias.mem	16 int32 bias values
layer3_weights.mem / bias.mem	16 int8 weights and one int32 bias
quant_params.json	Input/output scales and zero-points plus per-layer or per-channel requantization constants
golden_vectors.mem	Raw int8 request and expected response pairs

The integer arithmetic contract

A quantized integer q represents a real value using:

```
real_value = scale x (q - zero_point)
```

For output neuron j, the dense accumulator is:

```
acc[j] = bias[j] + sum_i ((qx[i] - zx) x (qw[j,i] - zw[j]))
```

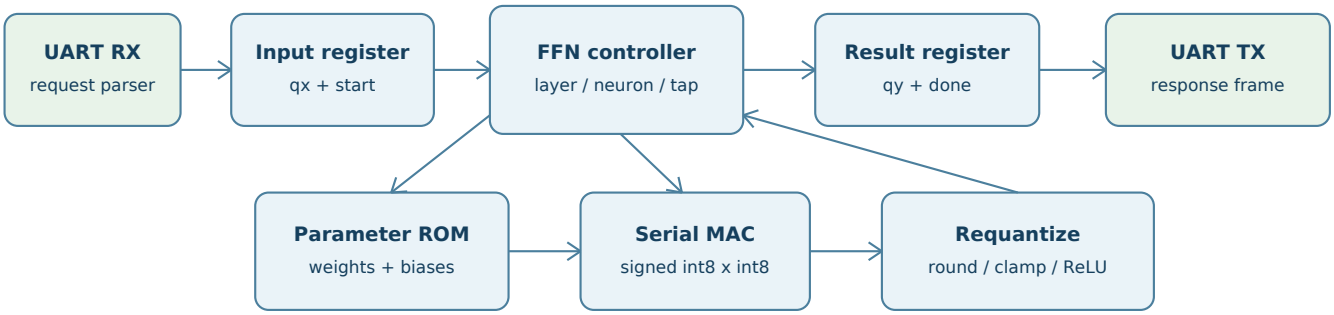
The accumulator is converted to the next tensor's int8 domain using a layer-specific or channel-specific multiplier, a signed shift, the output zero-point, rounding, and saturation. A fused ReLU raises the minimum output code to the code representing real zero.

Value	Required representation	Rule
Weights / activations	signed int8	Sign-extend before multiplication
Product	signed int16	Do not truncate before accumulation
Bias / accumulator	signed int32	Load the bias before the first product
Multiplier	signed fixed integer	Export using the same convention as the software reference
Layer output	signed int8	Round, add zero-point, apply activation limit, and saturate

Gate before RTL

Do not begin the inference controller until a standalone integer reference, reading the exported files, reproduces the TensorFlow Lite interpreter's raw int8 outputs for the golden vectors.

RTL module architecture



One physical MAC is reused across 288 logical multiplications.

Module	Required responsibility
parameter_rom	Present signed weights, signed biases, and requantization constants from generated initialization files.
dense_mac	Load an int32 bias, accept signed operand pairs, accumulate products, and expose the final int32 sum.
requantize_relu	Apply the exported multiplier/shift convention, round, add the output zero-point, apply optional ReLU, and saturate to int8.
ffn_controller	Sequence layers, output neurons, tap indices, ROM addresses, accumulator clearing, activation writes, and done timing.
sine_ffn	Provide the model-level start/busy/done interface and hold one signed int8 result.
uart_model_bridge	Parse the request frame, launch inference only when idle, and transmit the response after done.

Recommended model-level interface

Signal	Direction	Width	Behavior
clk	input	1	100 MHz system clock
rst_n	input	1	Active-low synchronous reset
start	input	1	One-cycle request; accepted only when busy = 0
x_q	input	8	Raw signed int8 model input
busy	output	1	High from accepted start until result completion
done	output	1	One-cycle pulse when y_q becomes valid
y_q	output	8	Raw signed int8 model output; held until the next result

Baseline architecture

Use one physical multiplier and accumulator. Parallel lanes are a later optimization, not a requirement. The baseline intentionally exposes the controller, memory-addressing, and quantization work hidden by software frameworks.

Inference sequencing

A complete forward pass contains 288 trained-weight multiplications. The controller must repeat a regular neuron procedure while switching tensor sources and layer-specific quantization constants.

Phase	Controller behavior	Products
Idle / capture	Wait for start while not busy; latch x_q and assert busy	0
Dense 1	For each of 16 neurons: load bias, multiply the scalar input by one weight, requantize, apply ReLU, store activation	16
Dense 2	For each of 16 neurons: load bias, accumulate 16 activation/weight pairs, requantize, apply ReLU, store activation	256
Dense 3	Load the final bias, accumulate 16 activation/weight pairs, requantize without ReLU, store y_q	16
Complete	Deassert busy and pulse done for exactly one system-clock cycle	0

Neuron procedure

1. Select the current layer and output-neuron index.
2. Read and sign-extend the neuron's int32 bias; load it into the accumulator exactly once.
3. For each input tap, address the correct activation and weight, then accumulate the signed product.
4. After the last tap, pass the stable accumulator through the layer's requantization path.
5. Apply the fused activation limit and store the int8 result in the destination activation bank.
6. Advance to the next neuron or layer without clearing values that are still needed as source activations.

Address and latency rules

- Document the weight address equation for every layer. For neuron-major Dense 2 storage, a typical mapping is `address = neuron x 16 + tap`; the exporter and RTL must agree.
- If ROM data appears one cycle after the address, pipeline the associated activation, valid flag, and last-tap indication by the same amount.
- Requantization may be combinational or multi-cycle, but its valid timing must be explicit and identical for all layers.
- Report kernel latency from accepted start to done. Do not include the UART receive or transmit time in that number.

Expected cycle count

The lower bound is 288 multiplication cycles, but bias loads, ROM latency, requantization, activation writes, and state transitions add overhead. Your documented controller schedule - not the theoretical lower bound - defines the expected latency.

UART host-to-model interface

Reuse a verified 115200-baud, 8-N-1 UART receiver and transmitter. The UART wrapper transports raw int8 tensor codes; conversion between radians and model codes remains on the computer.

Direction	Byte 0	Byte 1	Meaning
PC -> FPGA	0x53 ('S')	qx	Start marker followed by one raw two's-complement int8 input
FPGA -> PC	0x59 ('Y')	qy	Result marker followed by one raw two's-complement int8 output

UART bridge behavior

1. While idle, discard received bytes until the 0x53 start marker is observed.
2. Treat the next received byte as qx, preserve its bit pattern, and issue a one-cycle start pulse when the FFN core is idle.
3. Do not accept another request while inference or response transmission is active.
4. After done, transmit 0x59 and then qy. Wait for the UART transmitter to become ready between bytes.
5. Return to the start-marker state after the second response byte completes.

Host application behavior

For a requested angle x in radians, the computer performs:

```
qx = clamp(round(x / Sx) + Zx, -128, 127)
```

```
y_model = Sy x (qy - Zy)
```

Host step	Required action
Input	Accept x only within the model's documented input interval
Quantize	Use the input scale Sx and zero-point Zx from the locked model manifest
Transmit	Send [0x53, qx] as two raw bytes; do not send ASCII decimal text
Receive	Wait for [0x59, qy], then interpret qy as signed int8
Display	Show x, FPGA y_model, TensorFlow Lite y_model, and mathematical sin(x)

Timing distinction

At 115200 baud, a two-byte request and two-byte response take far longer than the model's arithmetic at 100 MHz. UART is the demonstration interface; the start-to-done cycle count is the accelerator measurement.

Design requirements

- Use the 100 MHz board clock as the only design clock. Generate enables for slower events; do not create a fabric-divided clock.
- Declare all weights, activations, products, biases, accumulators, multipliers, shifts, and comparisons with explicit widths and signedness.
- Initialize parameter memories only from files generated from the locked model. Do not manually retype trained values into RTL.
- Reset control state, valid flags, counters, busy, done, and the UART packet parser to deterministic values.
- Preserve model tensor ordering and per-layer or per-channel quantization. Never assume that every zero-point is zero or every scale is shared.
- Match the standalone integer reference at the raw int8 output. A visually sine-shaped curve is not sufficient if byte-level results differ.
- Keep the inference core independent of UART by using the start/busy/done interface defined in this assignment.
- Document the realized latency, weight address mapping, accumulator width, and requantization convention alongside the RTL.

Required project artifacts

Area	Artifact
Model setup	Locked int8 model identity, manifest, exported parameter files, and golden vectors
Software	Model inspection/export utility, standalone integer reference, and UART host application
RTL	Parameter ROM, dense MAC, requantization/ReLU, FFN controller, model wrapper, and UART bridge
Design record	Module interfaces, memory maps, cycle schedule, quantization convention, and kernel latency

Design scope

Included

Pretrained-model inspection and export; exact int8 forward inference; one serial MAC; constant parameter ROM; ReLU; UART request/response framing; host-side quantization and dequantization.

Outside this assignment

Model training or fine-tuning; changing the layer sizes; floating-point hardware; HLS; AXI; DMA; cameras or microphones; multiple simultaneous requests; parallel MAC lanes; on-board plotting; approximation with a lookup table, CORDIC, or polynomial instead of the FFN.

Authoritative model resources

TensorFlow Lite Micro, Hello World example - model purpose, evaluation flow, and example documentation.

TensorFlow Lite Micro, Hello World model files - official float and int8 pretrained files.

TensorFlow Lite Micro, training-source topology - confirms the 16-unit hidden layers and activations; training is not required here.

Google AI Edge, TensorFlow Lite Interpreter API - tensor details and quantization metadata.

Netron, model graph viewer - visual inspection of the fixed `.tflite` graph.

End of assignment

At completion, a real-valued angle entered on the computer must travel through the locked model's quantization, UART transport, synthesizable FFN datapath, and output dequantization to produce a repeatable sine approximation.